Field Tracking System

for Sales Executives

A multi-tenant, real-time field-staff tracking and management platform: a Python/Django REST + WebSocket backend, a Next.js web dashboard, and a Flutter mobile app for field staff.

Project report

Submitted by : _____

Roll number : _____

Guide : _____

Date : _____

Technologies: Python, Django, Django REST Framework, Django Channels, PostgreSQL + TimescaleDB, Redis, Next.js (React), TailwindCSS, Leaflet, Flutter, Docker, AWS.

1. Abstract

In today's fast-paced business environment, organizations rely heavily on field sales teams to drive revenue and maintain customer relationships. Managing and monitoring field executives efficiently remains a significant challenge due to the lack of real-time tracking and structured reporting. The Field Tracking System for Sales Executives addresses this with a comprehensive, mobile-first solution for tracking and managing field activities.

The mobile application is built with Flutter for a high-performance, cross-platform experience on Android and iOS. The backend is implemented in Python using the Django REST Framework, providing a secure, scalable, API-based architecture, with Django Channels adding real-time WebSocket communication. A Next.js web dashboard gives managers a live operational picture.

The system focuses on real-time location tracking of sales executives, daily activity logging, attendance (clock in/out), geofenced site check-ins, task dispatch with photographic proof of work, route playback, and performance analytics. Security and data integrity are maintained through JWT authentication, role-based access control, and strict per-tenant data isolation. The platform is containerized with Docker for reliable, repeatable deployment to the cloud.

2. Objectives

- Track the real-time location of field staff on a live map.
- Recor

3. Technology Stack

Layer	Technology
Backend API	Python, Django 5, Django REST Framework
Realtime	Django Channels (ASGI / Daphne) + Redis channel layer
Database	PostgreSQL 16 + TimescaleDB (location hypertable)
Live cache	Redis (latest position per worker, geofence state)
Authentication	JSON Web Tokens (djangorestframework-simplejwt)
Web dashboard	Next.js 14 (React), TypeScript, TailwindCSS, Leaflet
Mobile app	Flutter (Dart): geolocator, sqflite, image_picker
Maps & geo	OpenStreetMap tiles, Photon geocoding, OSRM routing
Deployment	Docker, Docker Compose, Nginx, AWS EC2

4. System Architecture

All clients communicate with a single Django service that speaks both HTTP (REST) and WebSockets (via Channels). Redis holds each worker's latest position for an O(1) live map read and powers real-time fan-out to dashboards. PostgreSQL stores relational data, with high-volume GPS history kept in a TimescaleDB hypertable. In production, Nginx fronts everything on one origin.

```
[Flutter mobile app]                [Next.js web dashboard]
      |  REST + WebSocket                  ^  REST + WebSocket
      v                                    |
  [ Django REST + Channels (Daphne, ASGI) ]-+
      |-> Redis        latest location, geofence state, fan-out
      |-> PostgreSQL   relational data + TimescaleDB location history
      '-> WebSocket    broadcast to tenant group:
                       location_sync . geofence_event . task_updated
```

Ping lifecycle

The mobile app buffers GPS pings in a local SQLite queue, then flushes them to the API. The server stores each batch, overwrites the worker's entry in the Redis live cache, diffs geofence containment to emit ENTER/EXIT events, and broadcasts updates to that tenant's dashboard group only. If the device is offline, pings wait in the queue and sync automatically when back online.

5. Multi-tenancy and Roles

The platform is multi-tenant: one deployment serves many organizations. Every tenant-owned record carries a tenant id, and the API scopes all queries to the caller's tenant (read from the verified JWT, never from the request), so one organization can never see another's data. Access is governed by four roles:

Role	Capabilities
Super Admin (owner)	Owner portal: create/suspend tenants, platform analytics, MRR
Tenant Admin	Manage team (invite, edit, reset passwords, roles, delete); all dispatcher rights
Dispatcher	Live map, route playback, task dispatch, geofences, performance
Field Staff	Mobile app: own tasks, clock in/out, location upload, proof capture

6. Modules and Features

Authentication & profile

JWT login with role-based redirection. Every user can manage their own profile and change their password; tenant admins can edit any member and reset passwords.

Real-time live map

Managers see every on-duty worker as a named avatar marker that moves in real time. A searchable staff finder lists all field staff; selecting one flies the map to their current location. Open task locations and geofences are overlaid.

Attendance & tracking

Workers clock in to start a shift; the app then tracks location in the background (foreground service) and stops on clock-out, clearing the live marker. Tracking only ever runs during an open shift, protecting battery and privacy.

Task dispatch & proof of work

Dispatchers create tasks and place them on the map (with worldwide place search). Field staff advance task status (assigned -> en route -> in progress -> completed), add notes, and attach photos captured with the device camera.

Geofencing

Circular site boundaries trigger automatic check-in / check-out events when a worker enters or leaves, producing an auditable visit log.

Route playback & analytics

Any worker's route for a chosen day can be replayed with total distance and an active/idle split. A performance report summarizes site visits, completed tasks, distance covered, active time, and hours on shift per staff member.

7. Application Screenshots

The following are screenshots of the running application.

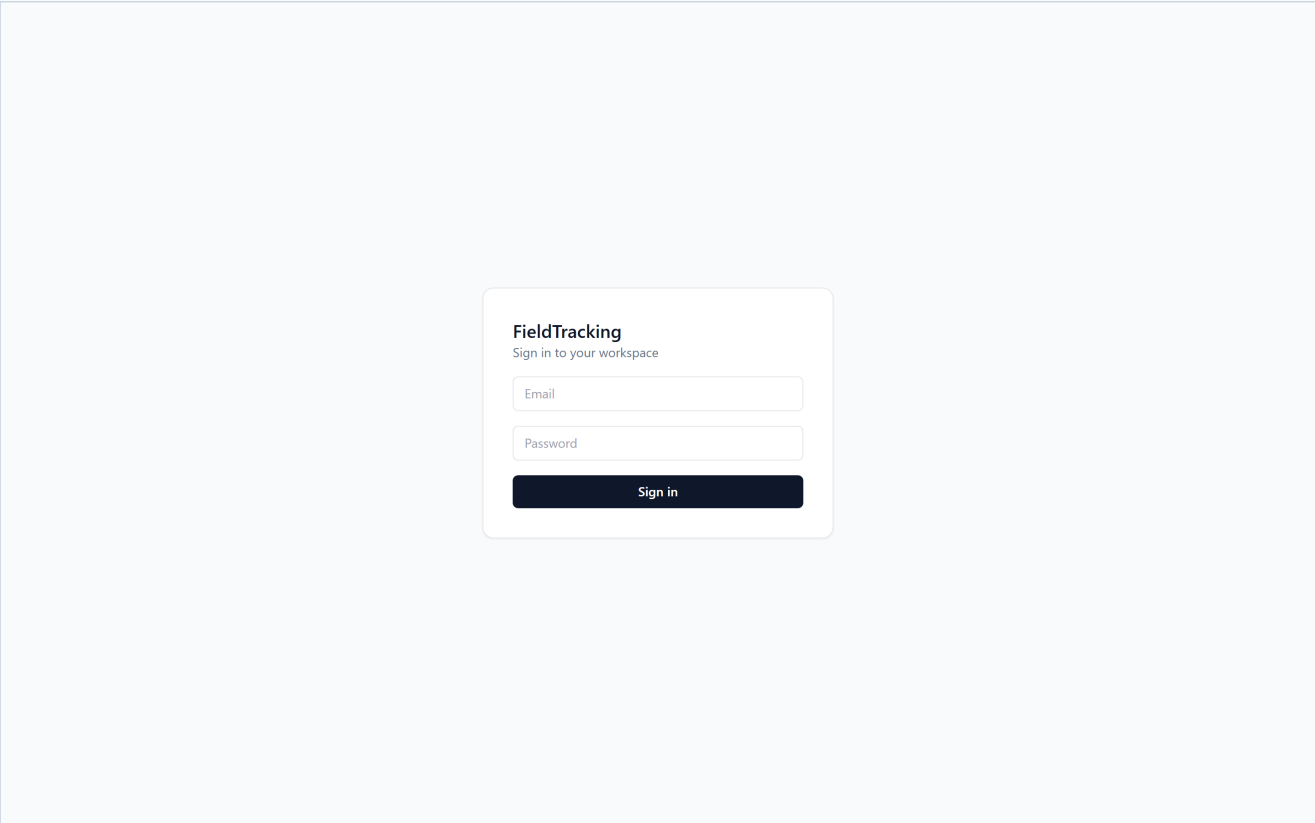


Figure 1: Secure login with role-based redirection.

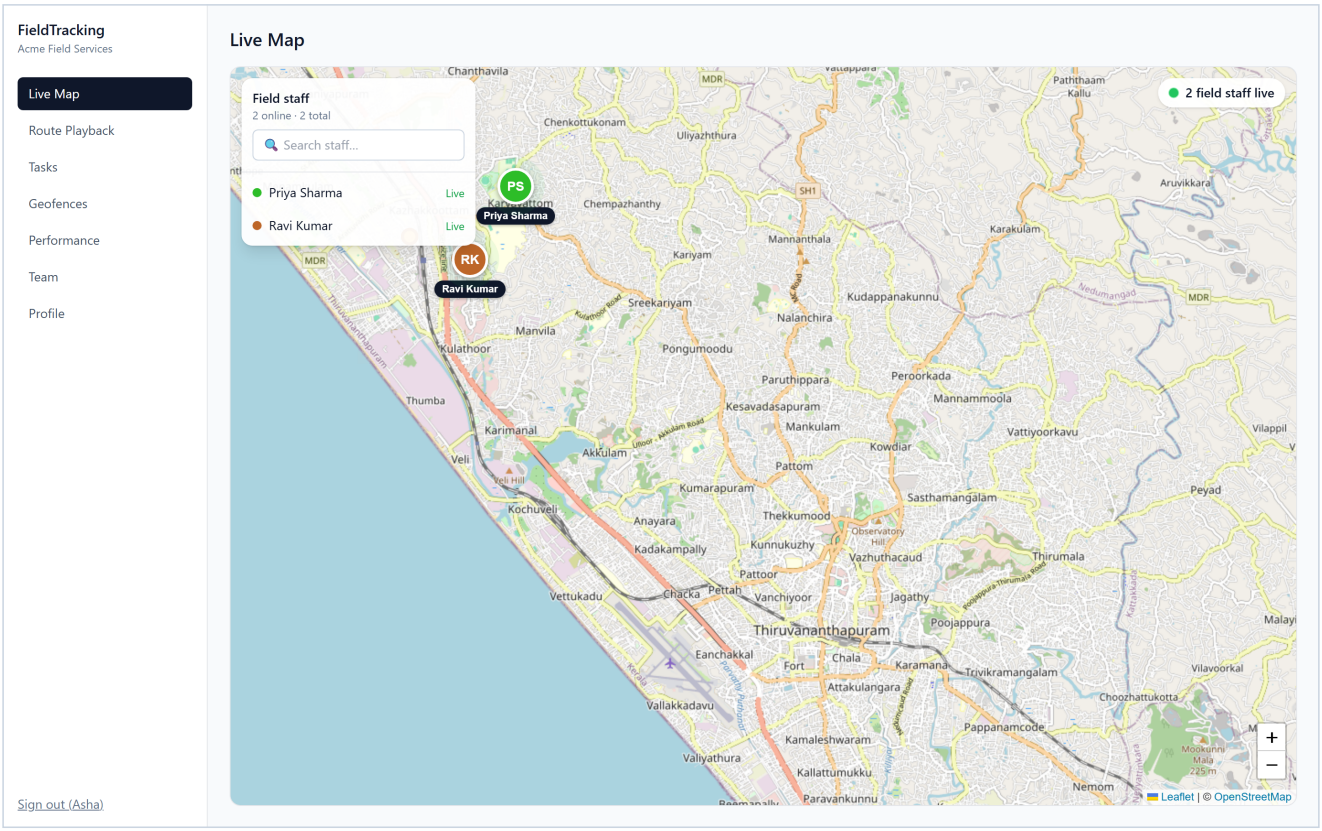


Figure 2: Real-time Live Map - avatar markers, searchable staff finder, and the live worker count.

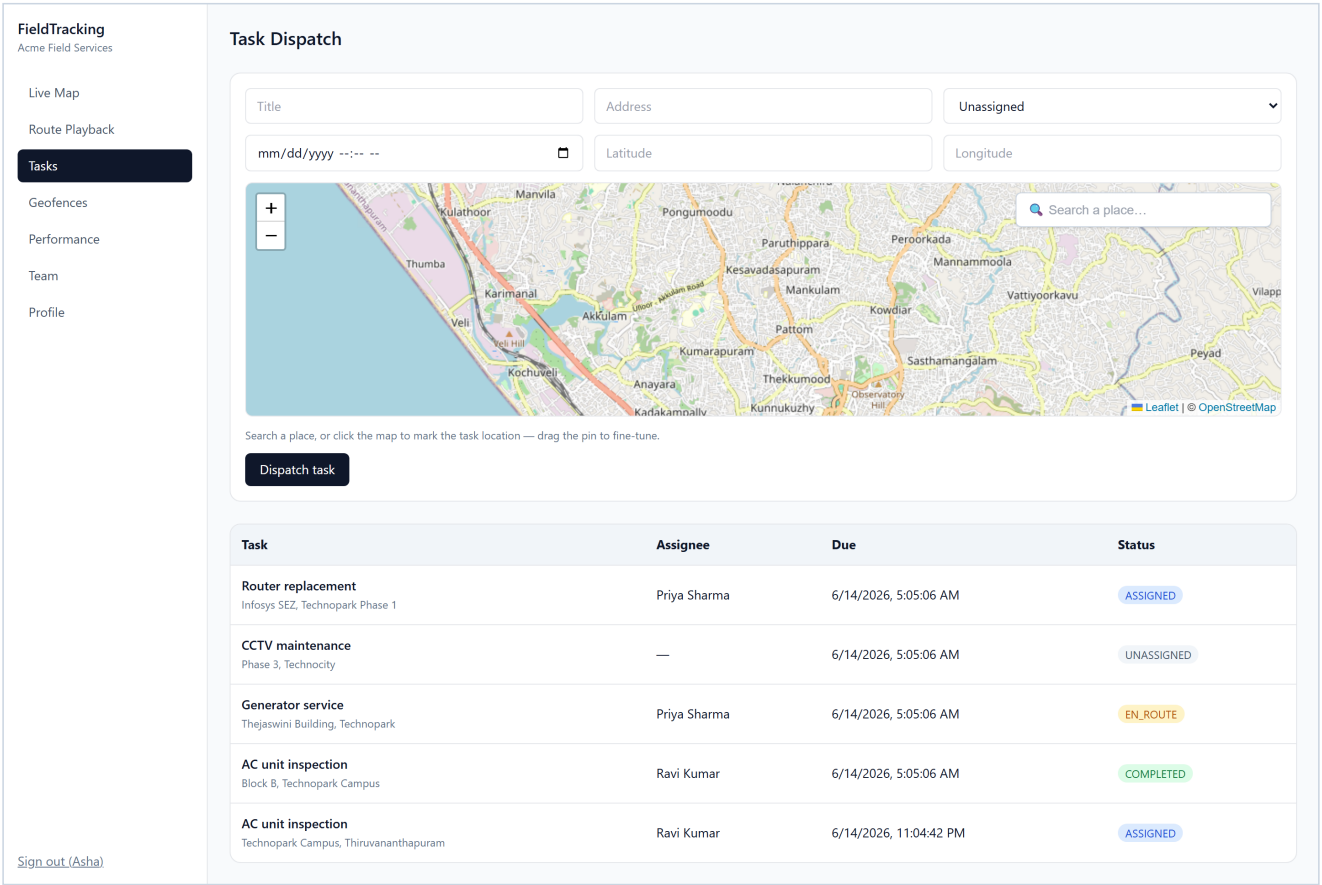


Figure 3: Task dispatch with map-based location picker and worldwide place search.

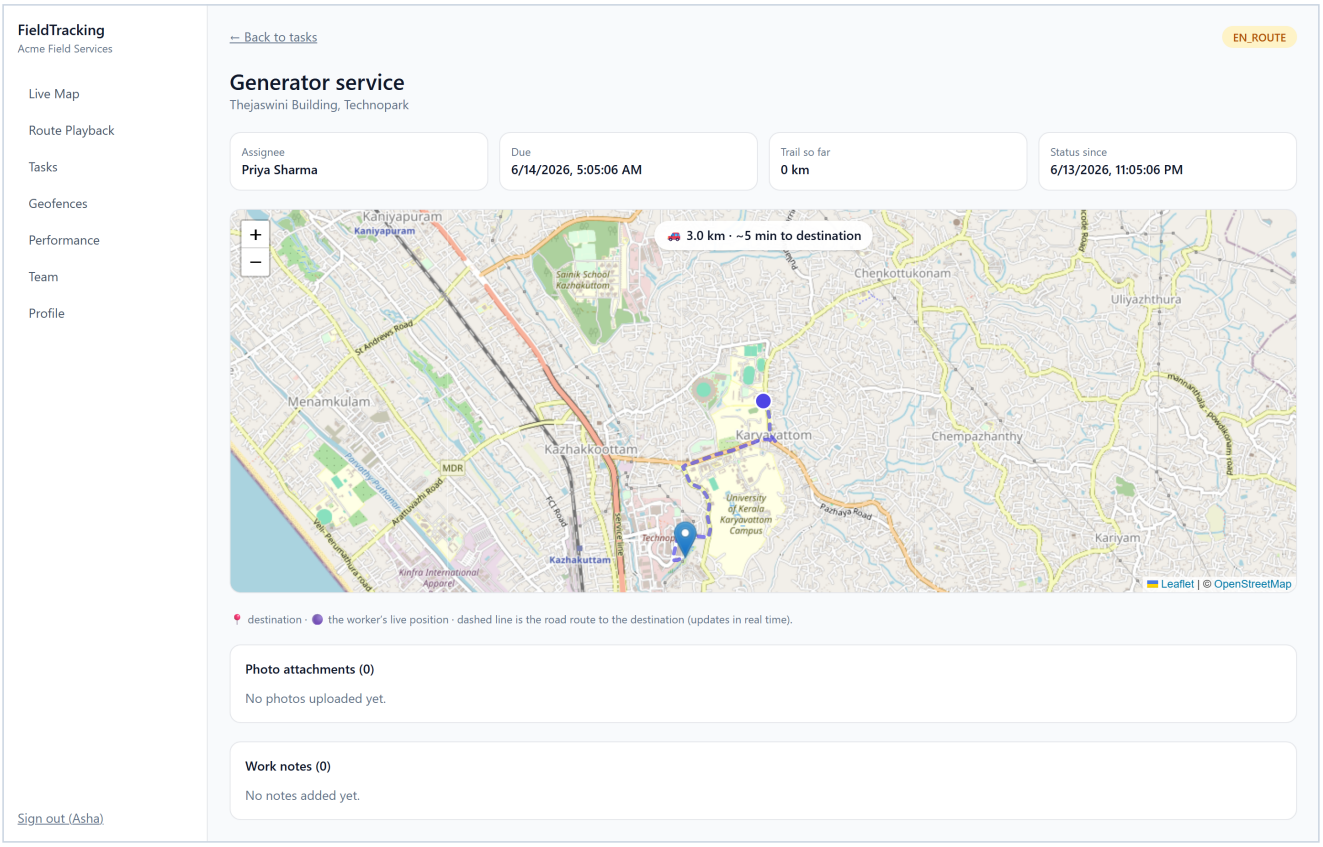


Figure 4: Active task - the worker's live position and the road route with ETA to the destination.

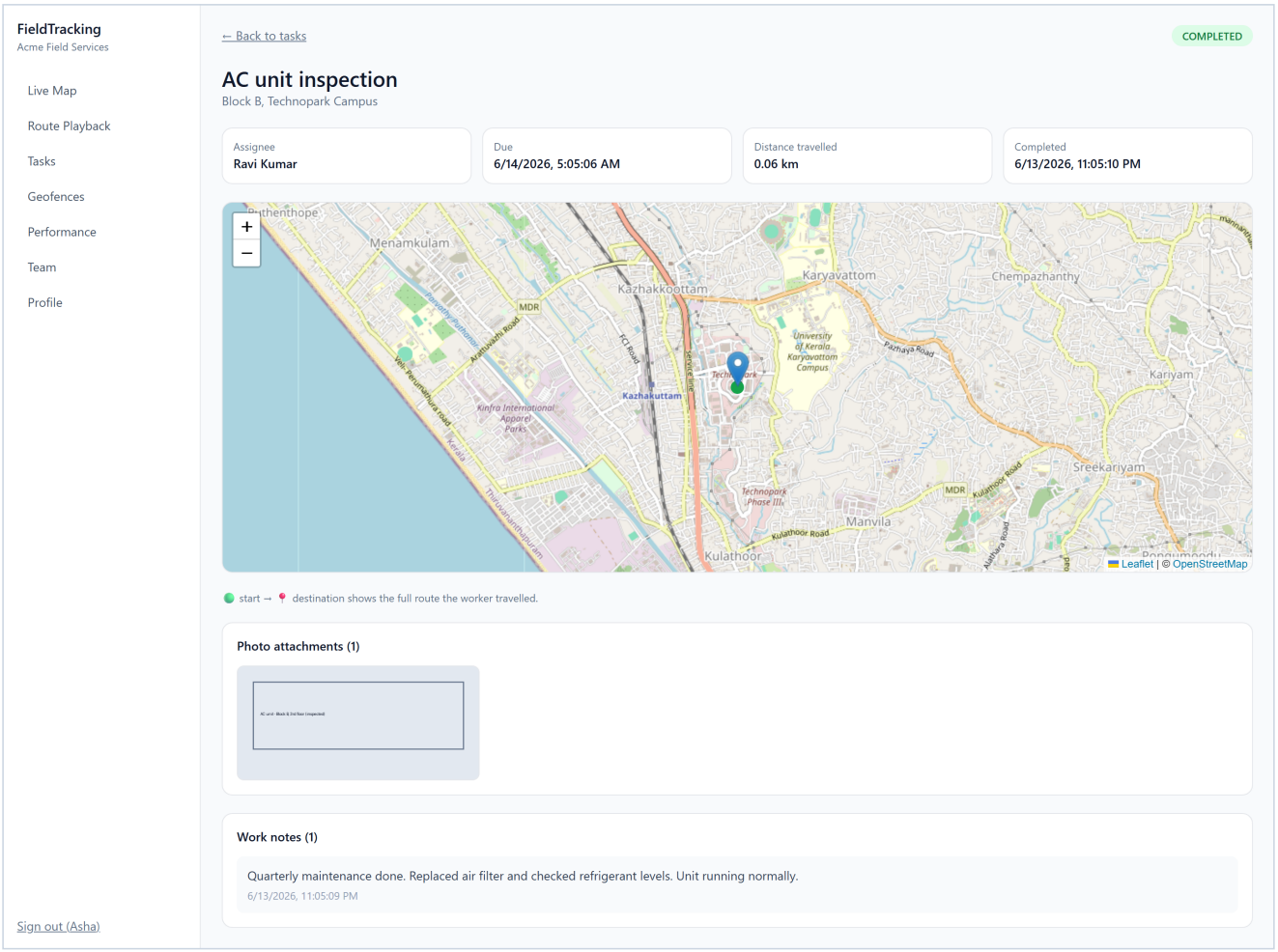


Figure 5: Completed task - travelled route, photographic proof of work, and work notes.

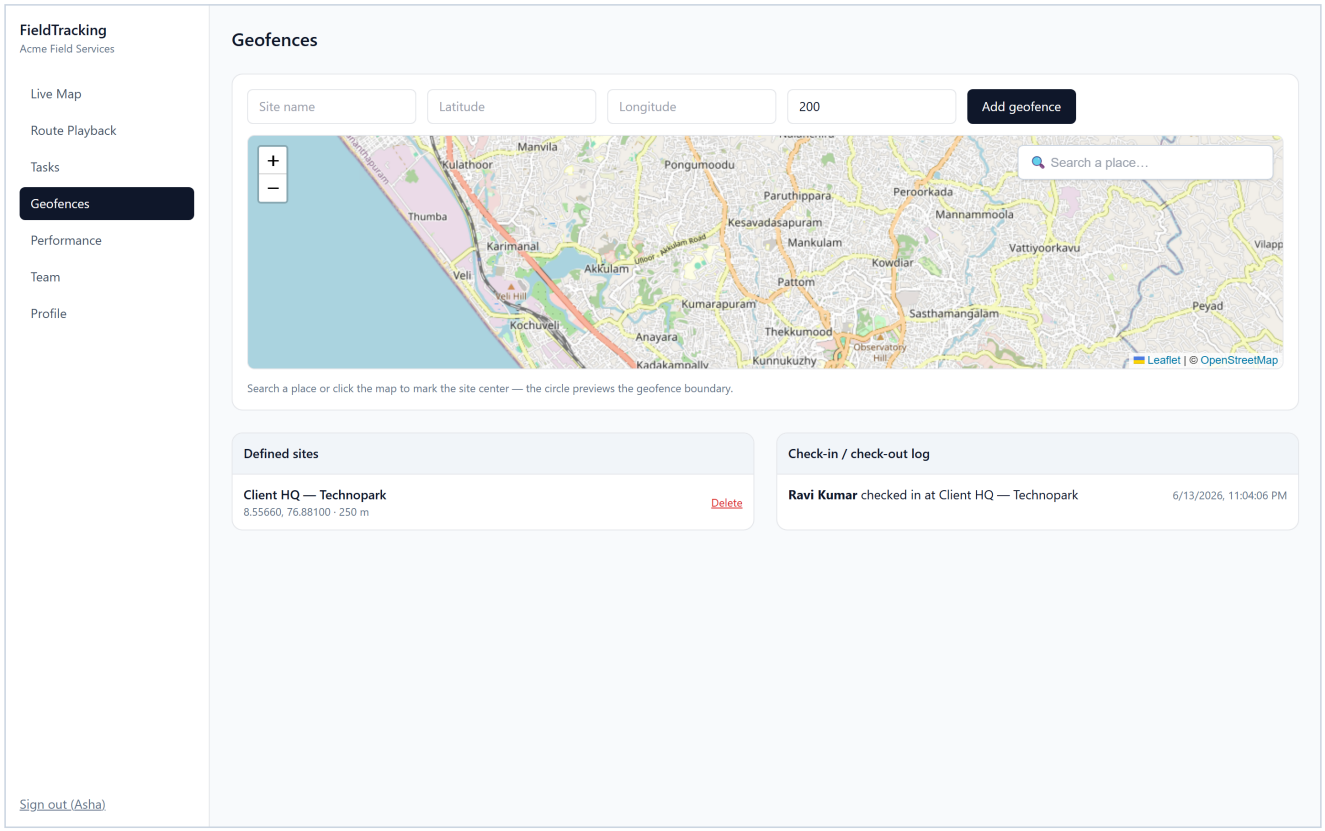


Figure 6: Geofence management and the check-in / check-out log.

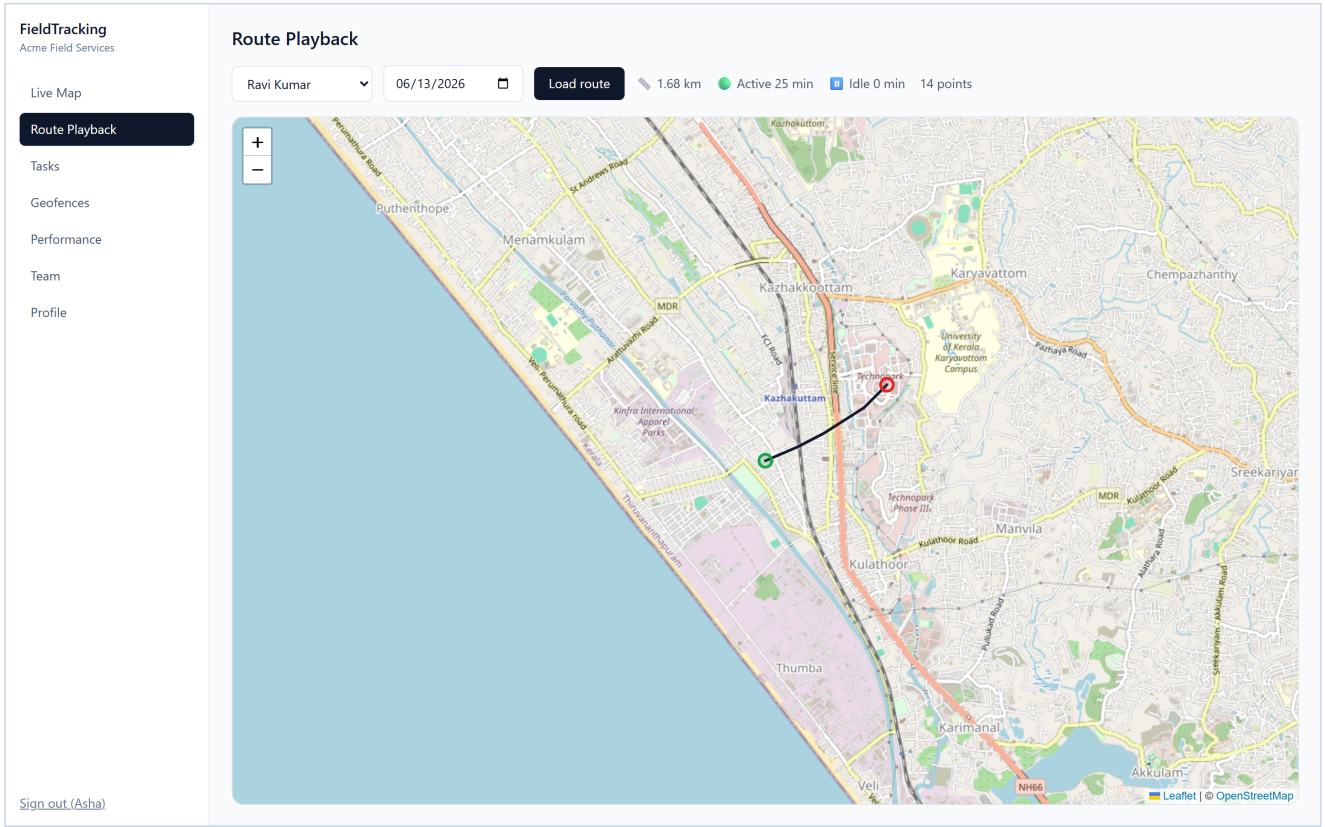


Figure 7: Route playback with distance and active/idle split.

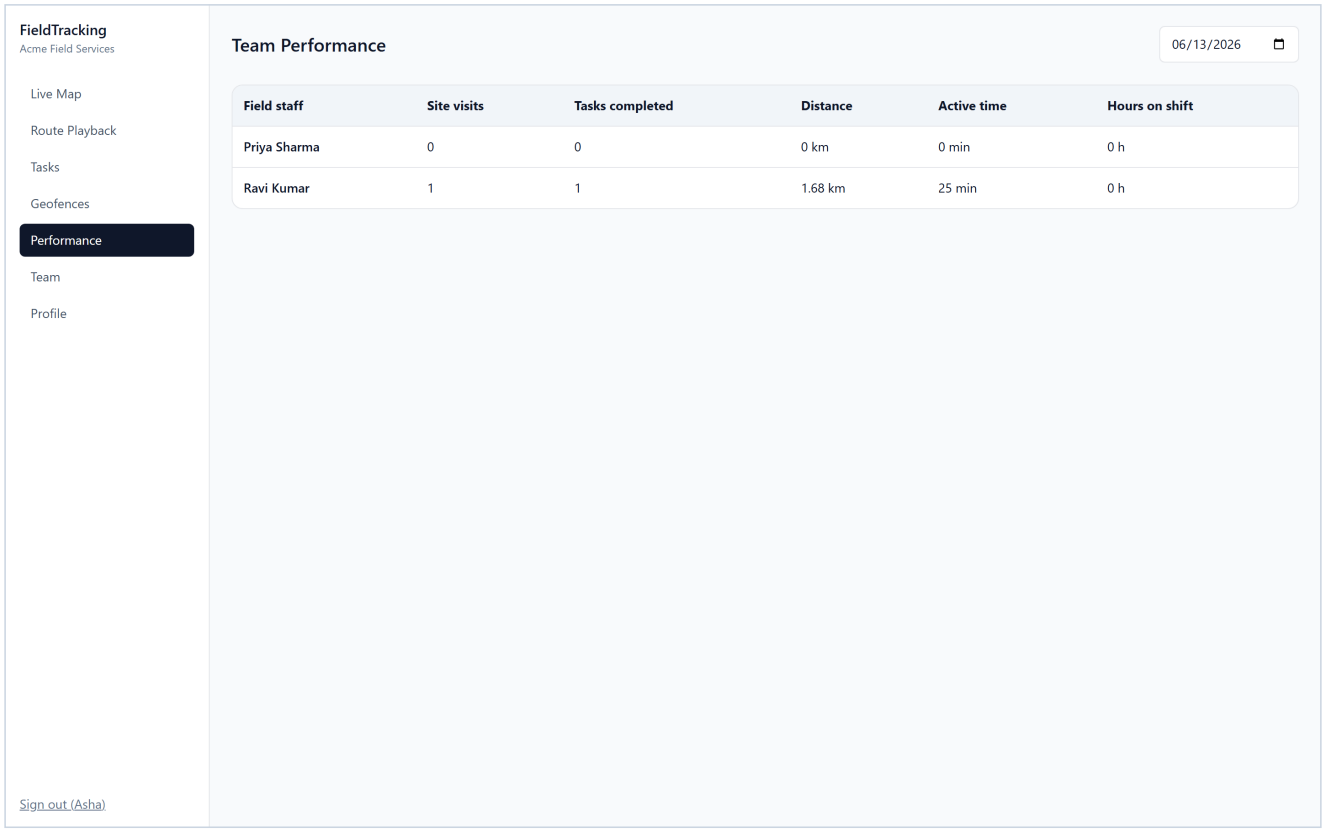


Figure 8: Per-staff performance analytics.

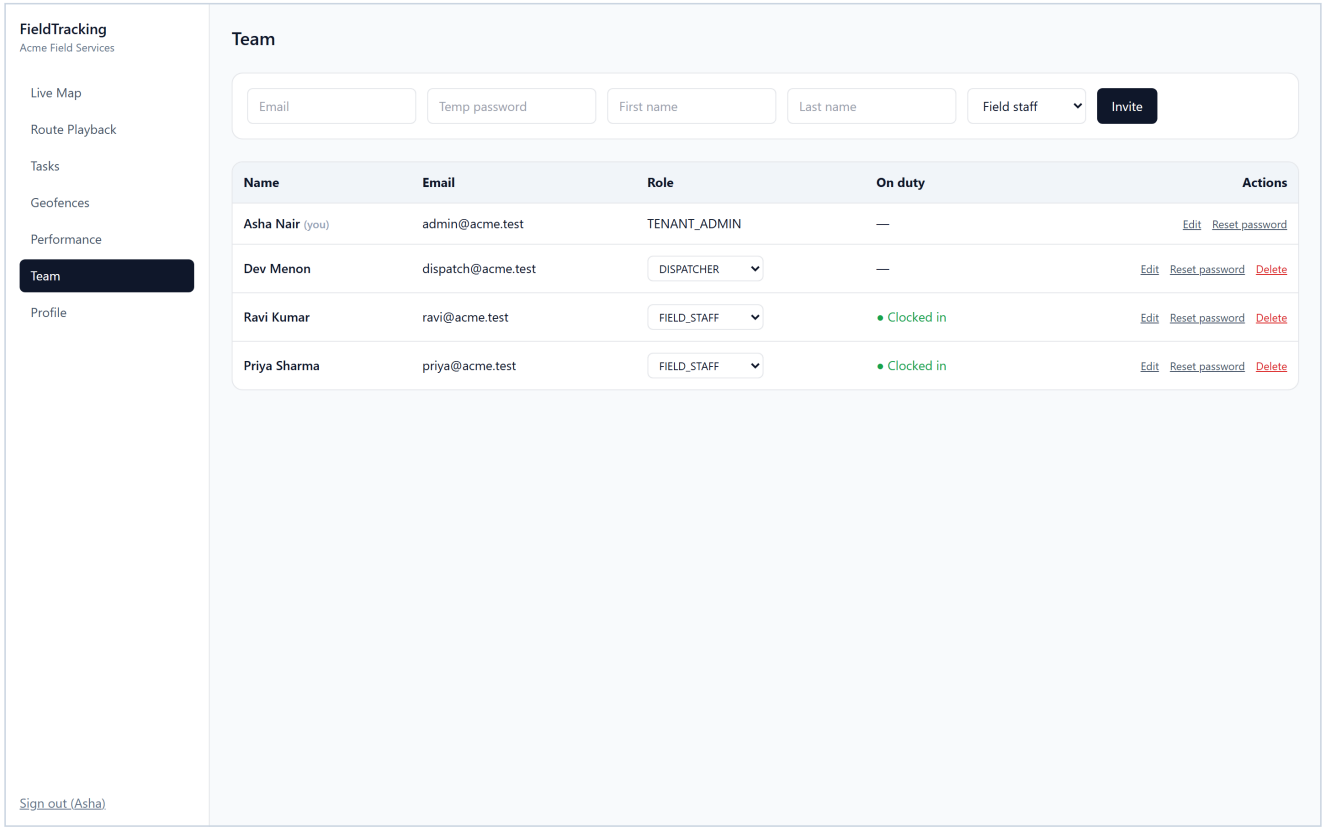


Figure 9: Team management - invite, edit, reset password, change role, and remove members.

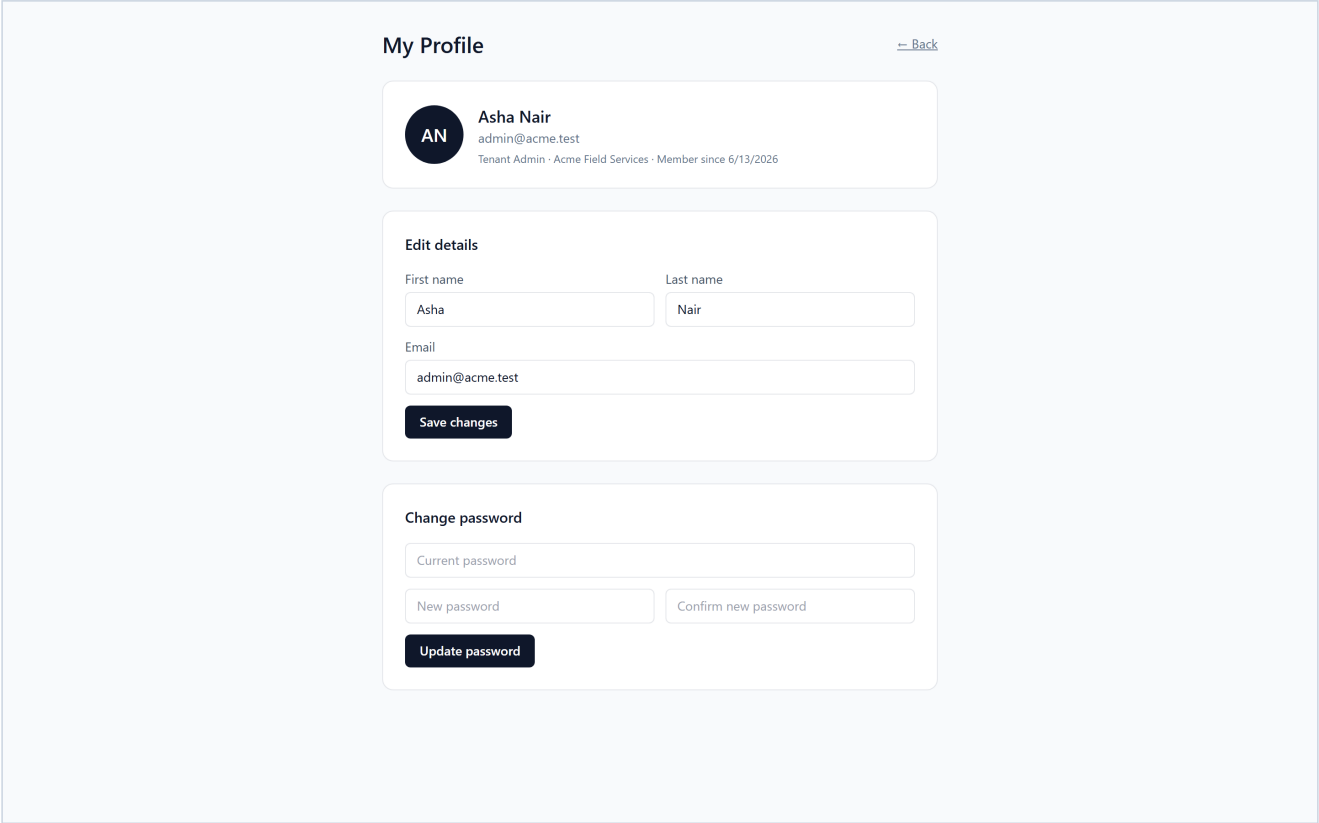


Figure 10: Self-service profile and password management.

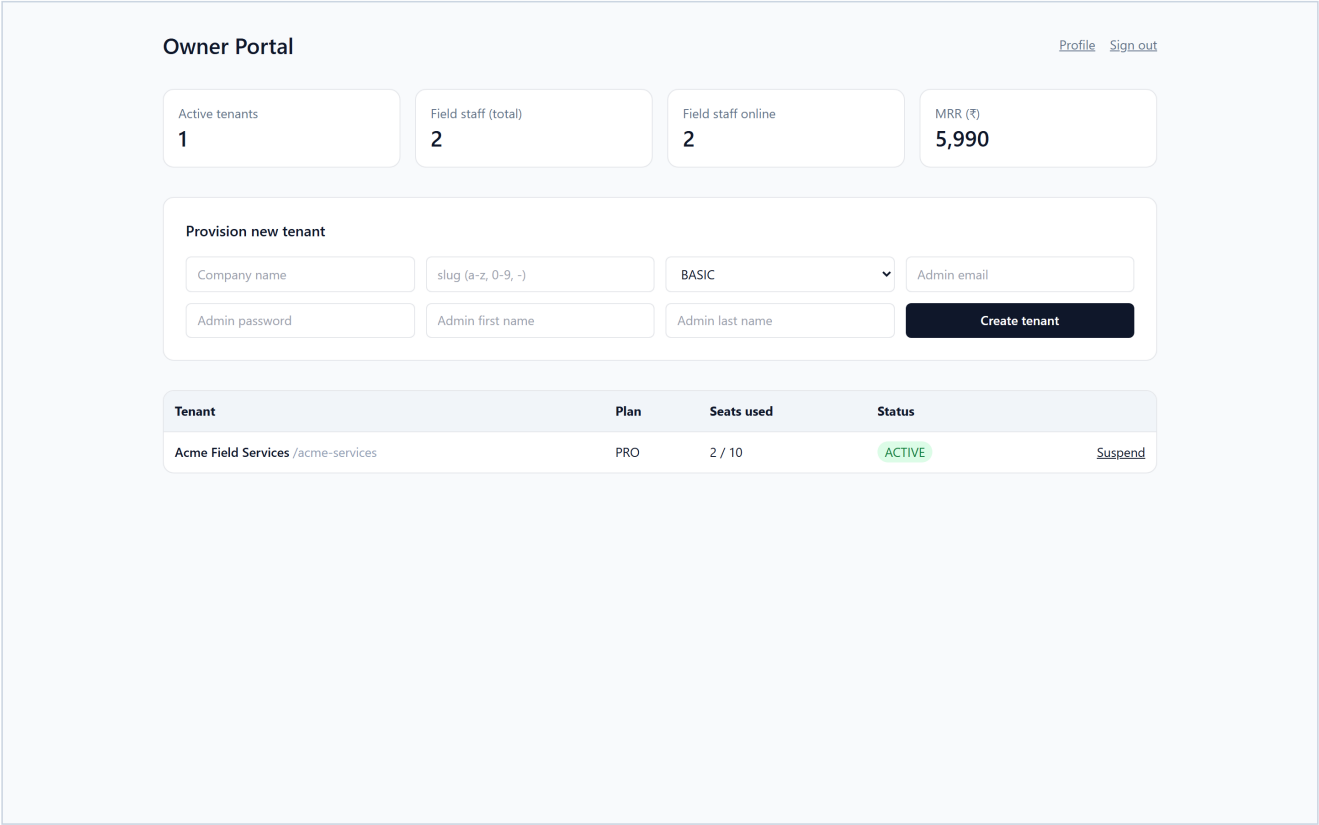


Figure 11: Platform owner portal - tenants, seats, and MRR.

8. Key API Endpoints

The REST API is versioned under /api/v1. A representative subset:

Method & path	Purpose
POST /auth/login	Authenticate, return JWT + user
GET /me . POST /me/password	Own profile / change password
GET /locations/live	Current positions of on-duty staff (live map)
POST /staff/locations	Upload a batch of GPS pings
GET /locations/history	Route playback for a user and date
GET/POST /tasks . GET /tasks/{id}/route	Dispatch board / task route + live
PATCH /staff/tasks/{id} . /photo	Advance status / attach camera photo
GET/POST /geofences . /geofences/events	Geofences and visit log
GET /reports/performance	Per-staff productivity report
GET/POST /owner/tenants	Owner: list / provision tenants

Interactive API documentation (Swagger UI) is served at /api/docs/.

9. Security

- JWT-based stateless authentication; passwords hashed with Django's hasher.
- Role-

10. Testing

The backend is covered by an automated pytest suite (40+ tests) spanning authentication, role permissions, tenant isolation, attendance, location ingestion and the live cache, geofence ENTER/EXIT detection, route history and performance, task dispatch and proof upload, profile and admin user management, and the real-time broadcast payload. The web app builds clean under the Next.js production compiler and the Flutter app passes static analysis.

11. Deployment

The whole platform is containerized. A development stack starts with a single command; a production stack adds an Nginx reverse proxy that serves the web app, REST API, WebSockets, admin, and uploaded media on one origin.

```
docker compose -f docker-compose.prod.yml up -d --build
```

It is designed to run on a single AWS EC2 instance (with PostgreSQL/TimescaleDB and Redis as containers), and can later scale out to managed AWS services - Amazon RDS, ElastiCache, S3 and ECS Fargate - without code changes. A detailed, step-by-step hosting guide accompanies this report.

12. Future Enhancements

- Subscription billing (the data model and MRR are already in place).
- Push

13. Conclusion

The Field Tracking System provides a practical, end-to-end solution for managing field sales operations. By combining a robust Django backend, real-time communication, a modern web dashboard, and a cross-platform Flutter app, it improves transparency, accountability, and decision-making. Its secure, multi-tenant, containerized architecture makes it reliable to operate and ready to grow - a demonstration of modern application development applied to a real-world business problem.